

Mathematics practicles

Q.1 Write python program to flip few bits randomly in BSC channel

```
import random
bits=[1,0,1,1,0]
p=0.6
received=[]
for bit in bits:
    r=random.random()
    if r<p:
        received.append(1-bit)
    else:
        received.append(bit)
print("Original Bits:", bits)
print("Received Bits:",received)
```

Q.2 TO Find probability

```
import random
bits=[1,0,1,1,0]
received=[1,0,1,1,0]
error=0
for i in range(len(bits)):
    if bits[i]!=received[i]:
        error=1
p=error/len(bits)
print(p)
```

Q.3 Write python program to BSC with Short Messege

```
import random
m="A"
p=0.4
received=[]
bits=[1,0,0,0,0,0,1]
for bit in bits:
    r=random.random()
    if r<p:
        received.append(1-bit)
    else:
        received.append(bit)
print("Original Bits:", bits)
print("Received Bits:",received)
```

Q.4 if flip the bit then print error dectect otherwise no error

```
import random
bits=[1,1,1,0,0,0]
p=0.3
received=[]
for bit in bits:
    r=random.random()
    if r<p:
        received.append(1-bit)
    else:
        received.append(bit)
print("Original Bits:", bits)
print("Received Bits:",received)
error=0
```

```

for i in range(len(bits)):
    if bits[i]!=received[i]:
        error=1
print(error)
if error==0:
    print("No Error")
else:
    print("Error detect")

```

Q.5 To flip bit Randomly if bit flip then error detect and print

```

probability
import random
bits=[1,1,1,0,0,0]
p=0.3
received=[]
for bit in bits:
    r=random.random()
    if r<p:
        received.append(1-bit)
    else:
        received.append(bit)
print("Original Bits:", bits)
print("Received Bits:",received)
error=0
for i in range(len(bits)):
    if bits[i]!=received[i]:
        error=1
print(error)
if error==0:
    print("No Error")
else:
    print("Error detect")
p=error/len(bits)
print(p)

```

Practicle 2

Maximum Likelihood Decoding MLD Method

*** Steps:**

1. First we have original bits ,received bits and probability.
2. Calculate forward probability of each pair of code book and received code word.
3. Apply MLD & Compare forward probability.
4. It gives maximum likely code word from given code book

Q.1 Find forward probability

```

code_word=[1,0,1]
received=[1,1,0]
p=0.1
forward_prob=1
for i in range(3):
    if code_word[i]==received[i]:
        forward_prob*=(1-p)
    else:

```

```

forward_prob*=p
print("code word:" , code_word)
print("Received code:" , received)
print("Forward probability:", forward_prob)

```

Q.2 Give python code to find forward probability for following code book

```

code_book=[[1,0,1],[0,0,0],[1,1,1]]
received=[1,1,0]
p=0.1
for code_word in code_book:
    forward_prob=1
    for i in range(3):
        if code_word[i]==received[i]:
            forward_prob*=(1-p)
        else:
            forward_prob*=p
    print("code Word:", code_word, "-> forward probability:",
forward_prob)
Output:
code Word: [1, 0, 1] -> forward probability: 0.0090000000000000001
code Word: [0, 0, 0] -> forward probability: 0.0090000000000000003
code Word: [1, 1, 1] -> forward probability: 0.081000000000000002

```

Q.3 Give Python code to find most likely received code word

```

code_book=[[1,0,1],[0,0,0],[1,1,1]]
received=[1,1,0]
p=0.1
best_prob=-1
for code_word in code_book:
    forward_prob=1
    for i in range(3):
        if code_word[i]==received[i]:
            forward_prob*=(1-p)
        else:
            forward_prob*=p
    print("code Word:", code_word, "-> forward probability:",
forward_prob)
    if forward_prob > best_prob:
        best_prob = forward_prob
        best_codeword=code_word
print("Received message:", received)
print("MLD choose code word:",best_codeword)
output:
code Word: [1, 0, 1] -> forward probability: 0.0090000000000000001
code Word: [0, 0, 0] -> forward probability: 0.0090000000000000003
code Word: [1, 1, 1] -> forward probability: 0.081000000000000002
Received message: [1, 1, 0]
MLD choose code word: [1, 1, 1]

```

Q.4 Given Python code to find most likely received code word

```

code_book=[[1,0,1],[1,0,0],[1,1,1]]

```

```

received=[1,1,0]
p=[0.1,0.2,0.3]
best_prob=-1
best_codeword=None
for code_word in code_book:
    prob=1
    for i in range(len(received)):
        if code_word[i]==received[i]:
            prob*=(1-p[i])
        else:
            prob*=p[i]
    print("code Word:", code_word, "-> forward probability:", prob)
    if prob > best_prob:
        best_prob = prob
        best_codeword=code_word
print("Received message:", received)
print("MLD choose code word:",best_codeword)
output:
code Word: [1, 0, 1] -> forward probability: 0.054000000000000006
code Word: [1, 0, 0] -> forward probability: 0.126
code Word: [1, 1, 1] -> forward probability: 0.21600000000000003
Received message: [1, 1, 0]
MLD choose code word: [1, 1, 1]

```

Math Pratical 3

Q.1 Convert Character to ASCII Code

```

ch='A'
a=ord(ch)
print(f"ASCII value of {ch} is {a}")

```

Q.1 Give Character from user and convert Character to ASCII Code.

```

ch=input("Enter Character:")
a=ord(ch)
print(f"ASCII value of {ch} is {a}")
Output
Enter Character: B
ASCII value of B is 66

```

Q.2 Convert ASCII Code to Character

```

code=int(input("Enter a ASCII Code"))
ch =chr(code)
print(f"Character of {code} is {ch}")
Output:
Enter a ASCII Code 66
Character of 66 is B

```

Q.3 How do you convert a single character into its 8 bit binary code representation

```

ch=input("Enter Character:")
Ascii=ord(ch)
print(Ascii)

```

```

binary=format(Ascii, '08b')
print(f"Binary Number of {ch} is {binary}")
Output:
Enter Character: D
68
Binary Number of D is 01000100

```

Q.4 How do you convert 8 bit binary code into its Single Character

```

binary=input("Enter a Binary code: ")
Asc=int(binary,2)
ch=chr(Asc)
print(f"Character of {binary} is {ch}")
Output:
Enter a Binary code: 01000100
Character of 01000100 is D

```

Q.5 How do you convert string of character into a string of 8 bit binary code

```

a=input("Enter a string")
for ch in a:
    asc=ord(ch)
    binary=format(asc, '08b')
    print(f"Binary number of {ch} is {binary}")
Output:
Enter a string AB
Binary number of A is 01000001
Binary number of B is 01000010

```

Q.6 How do You convert a string of 8 bit binary codes into strings of characters representation

```

a=int(input("enter range of binary number:"))
str1=''
for i in range(a):
    binary=input("Enter a string of binary code")
    Asc=int(binary,2)
    ch=chr(Asc)
    str1+=ch
print(f"Character of {binary} is {str1}")
Output:
enter range of binary number: 2
Enter a string of binary code 01000001
Enter a string of binary code 01000010
Character of 01000010 is AB

```

Math Pratical 4

Implement a Binary Symmetric Channel(BSC) and Maximum likelihood decoding for a simple binary code

Q.1 Generating all possible words from a user define alphabate

```

import itertools
alphabate=input("Enter Alphabate: ")
all_word= list(itertools.product(alphabate,alphabate))

```

```

for word in all_word:
    print("".join(word))
print(f"Total Word generate: {len(all_word)}")
Output:
Enter Alphabate: ab
Enter Word Length: 2
aa
ab
ba
bb
Total Word generate: 4

```

Q.2 Generate & Analyse block codes over a user define alphabate in python.

```

import math
alphabate=input("Enter Alphabate: ")
q=int(input("Enter the Length of alphabate: "))
n=int(input("Enter the Length of code word: "))
m=int(input("Enter the Size of code word: "))
rate=math.log(m,q)/n
print(rate)

```

```

Output:Enter Alphabate: ab
Enter the Length of alphabate: 2
Enter the Length of code word: 2
Enter the Size of code word: 4
1.0

```

Q.3 Simulating a binary Symmetric Channel And calculating word transition probability(p) OR Find Forward Probability

```

code_word=input("Enter Sent Code word")
received=input("Enter Received word")
p=0.1
forward_prob=1
for i in range(3):
    if code_word[i]==received[i]:
        forward_prob*=(1-p)
    else:
        forward_prob*=p
print("code word:" , code_word)
print("Received code:" , received)
print(f"Forward probability: p({code_word}|{received}) = {forward_prob}")

```

```

Output:
Enter Sent Code word 110
Enter Received word 111
code word: 110
Received code: 111
Forward probability: p(110|111) = 0.081000000000000002

```

Q.4 Implementation of a Maximum Likelihood Decoder (MLD)

```

code_book=[[1,1,1],[0,0,0],[1,0,1]]

```

```

received=[1,1,0]
best_dist=4
best_codeword=None
for code_word in code_book:
    dist=0
    for i in range(len(received)):
        if code_word[i]!=received[i]:
            dist=1
    print("code Word:", code_word, "-> Hamming distance:",
dist)
    if dist < best_dist:
        best_dist = dist
        best_codeword=code_word
print("Received message:", received)
print("MLD choose code word:",best_codeword)
Output:
code Word: [1, 1, 1] -> Hamming distance: 1
code Word: [0, 0, 0] -> Hamming distance: 2
code Word: [1, 0, 1] -> Hamming distance: 2
Received message: [1, 1, 0]
MLD choose code word: [1, 1, 1]

```

Practicle 5

Burst Error Hamming Distance, and Nearest Neighbour

Decoding Method

â Burst Error :- It is a Type of error in digital communication where two or more bits in a row(consequently) are change during transmission.

Q.1 Simulate Burst Error on user define code book

```

code_book= input("Enter code word: ").split()
print(code_book)
for w in code_book:
    m=list(w)
    for i in [0,2]:
        if m[i]=='0':
            m[i]='1'
        else:
            m[i]='0'
    print(w , "-> ", ''.join(m) )

```

```

output =
Enter code word: 0000 1100 1010
['0000', '1100', '1010']
0000 -> 1010
1100 -> 0110
1010 -> 0000

```

Q.2 Compute the hamming distance between code words from user define code book

```

code_book= input("Enter code word: ").split()
print(code_book)

```

```

print("Hamming Distance of the given code_word")
for i in range(len(code_book)):
    for j in range(i, len(code_book)):
        d=0
        for k in range(len(code_book[i])):
            if code_book[i][k] != code_book[j][k]:
                d=1
        print(f"H {code_book[i], code_book[j]}:{ d}")
output =
Enter code word: 111 101 000
['111', '101', '000']
Hamming Distance of the given code_word
H ('111', '101'):1
H ('111', '000'):3
H ('101', '000'):2

```

Q.3 Find Minimum distance between two code word from user define code book

```

code_book= input("Enter code word: ").split()
print(code_book)
print("Hamming Distance of the given code_word")
for i in range(len(code_book)):
    for j in range(i, len(code_book)):
        d=0
        for k in range(len(code_book[i])):
            if code_book[i][k] != code_book[j][k]:
                d=1
        print(f"H {code_book[i], code_book[j]}:{ d}")
min_dist=5
if d<min_dist:
    min_dist=d
print("Minimum Distance",min_dist)
output =
Enter code word: 1111 0000 1010
['1111', '0000', '1010']
Hamming Distance of the given code_word
H ('1111', '0000'):4
H ('1111', '1010'):2
H ('0000', '1010'):2 Minimum Distance 2

```

Q.4 Apply Nearest neighbour decoding from user defined code book

```

code_book= input("Enter code word: ").split()
print(code_book)
received=input("Enter Received word")
min_dist=4
best_codeword=None
for cw in code_book:
    d=0
    for j in range(len(received)):
        if cw[j] != received[j]:
            d=1
    print(f"H {received, cw}:{ d}")
    if d<min_dist:

```

```

min_dist=d
best_codeword=cw
print("Nearest neighbour decoding result:")
print("Received word: ",received)
print("Decoded Code word:", best_codeword)
print("Distance:", min_dist)
output=
Enter code word: 110 001 111 000
['110', '001', '111', '000']
Enter Received word 101
H ('101', '110'):2
H ('101', '001'):1
H ('101', '111'):1
H ('101', '000'):2
Nearest neighbour decoding result:
Received word: 101
Decoded Code word: 001
Distance: 1

```

Practicle no 7

Q.1 Write a python Program to vary error probability

p=0.05,0.1,0.2,0.3

transmit the same message and count the number of error in each case

```

import random
msg="10101"
p_value=[0.05, 0.1,0.2,0.3]
run=10
for p in p_value:
    error=0
    for r in range(run):
        received=""
        for bit in msg:
            r=random.random()
            if r<p:
                if bit=='0':
                    received='1'
                else:
                    received='0'
            else:
                received=bit
        for i in range(len(msg)):
            if received[i]!=msg[i]:
                error=1
        print(f"{p:.2f}\t\t\t{error}")

```

Output:

```

0.05 1
0.10 0
0.20 1
0.30 1

```

Q.2 Write python program to plot a graph of error probability (xaxis) vs number of error (y-axis)

```

import matplotlib.pyplot as plt
#import random
msg="10101"
p_value=[0.0 ,0.1,0.2, 0.3,0.4,0.5]
run=10
list_error=[]
for p in p_value:
    error=0
    for r in range(run):
        received=""
        for bit in msg:
            r=random.random()
            if r<p:
                if bit=='0':
                    received='1'
                else:
                    received='0'
            else:
                received=bit
        for i in range(len(msg)):
            if received[i]!=msg[i]:
                error=1
        list_error.append(error)
print(list_error)
plt.plot(p_value,list_error,marker='o')
plt.xlabel("Error Probability")
plt.ylabel("Number Of error")
plt.title("Error Probability VS Number Of error")
plt.show()

```

Output

0, 1, 1, 1, 4, 2]

Q.3 Write a python program to generate random binary messege of length 10

transmit it with p=0.1 and count number of error

```

import random
msg="".join(random.choice("01") for _ in range(11))
p=0.1
received=""
error=0
for bit in msg:
    r=random.random()
    if r<p:
        if bit=='0':
            received='1'
        else:
            received='0'
    else:
        received=bit
for i in range(len(msg)):
    if received[i]!=msg[i]:
        error=1
print(f"{msg}\t\t{received}\t\t {error}")

```

Output:

01101010110 01101010110 0

Q.4 Write a python program to transmit message of different length (10,50,100,500) with fix $p=0.1$ and plot message length vs no.of error

```
import matplotlib.pyplot as plt
import random
error_list=[]
p=0.1
length=[10,50,100,500]
for i in length:
    msg="".join(random.choice("01") for _ in range(i))
    received=""
    error=0
    for bit in msg:
        r=random.random()
        if r<p:
            if bit=='0':
                received='1'
            else:
                received='0'
        else:
            received=bit

    for i in range(len(msg)):
        if received[i]!=msg[i]:
            error=1
    error_list.append(error)
    print(f"{msg} \t\t {error}")
plt.plot(length,error_list,marker='o')
plt.xlabel("length of message")
plt.ylabel("Number Of error")
plt.title("length of message VS Number Of error")
plt.show()
```

Basic of field and polynomial ring

Q.1 write a python program to perform addition,substraction, Multiplication and modular inverse of integer modulo a prime p, and display sample computation

```
p= int(input("Enter prime module p: "))
a=int(input("Enter first number a: "))
b=int(input("Enter first number b: "))
print("\n Result")
print("Addition" , (a) % p)
print("Multiplication" , (a*b) %p)
print("Substraction" , (a-b) %p)
inv_a =None
for x in range(1,p):
    if(a*x)%p==1:
        inv_a=x
        break;
print("Inverse of a =", inv_a)
```

```

inv_b=None
for x in range(1,p):
    if (b*x)%p==1:
        inv_b=x
        break;
print("Inverse of a =", inv_b)

```

Output:

```

Enter prime module p: 7
Enter first number a: 5
Enter first number b: 3
Result
Addition: 1
Multiplication: 1
Substraction: 2
Inverse of a = 3
Inverse of a = 5

```

Q.2 Write a python program to generate and print addition and multiplication table modulo a small prime.

```

p= int(input("Enter prime module p: "))
print(f"\nAddition Table mod {p}")
print(" ", end=" ")
for b in range (p):
    print(f"{b:2}", end = " ")
for a in range (p):
    print(f"{a:2}", end = " ")
    for b in range(p):
        print(f" { (a)%p:2}",end=" ")
    print()
print(f"\n Multiplication Table mod {p}")
print(" * ", end=" ")
for b in range (p):
    print(f"{b:2}", end = " ")
print()
for a in range (p):
    print(f"{a:2}", end = " ")
    for b in range(p):
        print(f" { (a*b)%p:2}",end=" ")
    print()

```

Output:

```

Enter prime module p: 5
Addition Table mod 5
 0 1 2 3 4
0 0 1 2 3 4
1 1 2 3 4 0
2 2 3 4 0 1
3 3 4 0 1 2
4 4 0 1 2 3

```

```

Multiplication Table mod 5
* 0 1 2 3 4
0 0 0 0 0 0
1 0 1 2 3 4

```

```

2 0 2 4 1 3
3 0 3 1 4 2
4 0 4 3 2 1

```

Q.3 Write a python program to test whether Z_n forms a field by checking if n is prime and report the result

```

n=int(input("enter n: "))
prime=True
if n<=1:
    prime=False
for i in range(2,n):
    if n%i==0:
        prime=False
        break
if prime:
    print(f" $Z_{\{n\}}$  from a field (since  $\{n\}$  is a prime).")
else:
    print(f" $Z_{\{n\}}$  does not from a field (since  $\{n\}$  is not a prime).")

```

Output:

```

Enter n: 5
 $Z_5$  from a field (since 5 is a prime).

```

Q.4) write a python program to represent polynomials F_p as list of coefficients and perform polynomial addition and subtraction modulo p .

```

p=int(input("Enter prime number p:"))
f=eval(input("Enter polynomial f as a list of coefficient:"))
q=eval(input("Enter polynomial q as a list of coefficient:"))
n=max(len(f), len(q))
f2=f[0]*(n-len(f))
q2=q[0]*(n-len(q))
add = [(f2[i][i])%p for i in range(n)]
sub = [(f2[i]-q2[i])%p for i in range(n)]
print("f(x) (x) mod", p,"=",add)
print("f(x)-q(x) mod", p,"=",sub)

```

Output:

```

Enter prime number p: 5
Enter polynomial f as a list of coefficient: [1,0,2,5]
Enter polynomial q as a list of coefficient: [2,2,0,3,6]
f(x) (x) mod 5 = [3, 2, 2, 3, 1]
f(x)-q(x) mod 5 = [4, 3, 2, 2, 4]

```

Q.5 Write a python program to multiply two polynomials over $\{a,b\}$ using coefficient list and reduce coefficient modulo p

```

p=int(input("Enter prime number p:"))
f=eval(input("Enter polynomial f as a list of coefficient:"))
g=eval(input("Enter polynomial q as a list of coefficient:"))
n=len(f)
m=len(g)
result=[0]*(n1)

```

```

for i in range(n):
    for j in range(m):
        result[i]=f[i]*g[j]
        result[i]%=p
print("f(x)*g(x) mod", p,"=",result)

```

Output:

```

Enter prime number p: 7
Enter polynomial f as a list of coefficient: [1,0,2]
Enter polynomial q as a list of coefficient: [2,2,0,3]
f(x)*g(x) mod 7 = [2, 2, 4, 0, 0, 6]

```

The Distance Of Code And Error Coorection Performance.

Q.1 write python program to simulate transmission error

```

import random
CW=input("Enter codebook seperated by space:").split()
original=random.choice(CW)
received=""
p=0.3
for bit in original:
    r=random.random()
    if(r<p):
        if bit=='0':
            received='1'
        else:
            received='0'
    else:
        received=bit
print("Code book:", CW)
print("Original: ", original)
print("Received:", received)
Output:
Enter codebook seperated by space: 000 111
Code book: ['000', '111']
Original: 111
Received: 011

```

Q.2 Find Error detection and Error Correction Capability

â Error Detection Capability: $d(\min)-1$

â Error Correction Capability: $d(\min)-1/2$

```

code_book= input("Enter code word: ").split()
print(code_book)
for i in range(len(code_book)):
    for j in range(i, len(code_book)):
        d=0
        for k in range(len(code_book[i])):
            if code_book[i][k]!= code_book[j][k]:
                d=1
min_dist=999
if d<min_dist:
    min_dist=d

```

```

print("Minimum Distance",min_dist)
print("Error Detection Capabiliy: ",min_dist-1)
print("Error Correction Capability: ",(min_dist-1)/2)
Output:
Enter code word: 000 111
['000', '111']
Minimum Distance 3
Error Detection Capabiliy: 2
Error Correction Capability: 1.0

```

Q.3 Analyse Error Coorrecting Capability.

```

code_book= input("Enter code word: ").split()
print(code_book)
original=input("Enter Original Word: ")
received=input("Enter Received word")
min_dist=4
best_codeword=None
for cw in code_book:
    d=0
    for j in range(len(cw)):
        if received[j]!= cw[j]:
            d=1
    if d<min_dist:
        min_dist=d
        best_codeword=cw
print("Original word: ",original)
print("Received word: ",received)
print("Decoded Code word:", best_codeword)
if(original==best_codeword):
    print("Error Corrected")
else:
    print("Error Detected")

```

```

Output:
Enter code word: 000 111
['000', '111']
Enter Original Word: 000
Enter Received word 101
Original word: 000
Received word: 101
Decoded Code word: 111
Error Detected

```

practical NO:8

1. Write a python program to perform addition, Subtraction, multiplication and modular inverse of integers module prime p, and display simple computatios

```

example:-
P=7
a = 5
b=3
P= int (input ("Enter prime modules P :"))1

```

```

a = int(input("Enter first integer a:"))
b = int(input("Enter Second integer b:"))
Print ("In resultS:")
Print ("Addition = " (a)% . P(
Print ("Substraction = " (a-b)% . P(
Print ("Multiplication =", (a * b) % . p)
inv a = None
for x in range (1, P):
if ca* x ) 7. P == 1:
inv a = X
break
Print ("Inverse of a = ", inv a)
inv b= None
for x in range (1,P):
if (b* x) % P == 1:
inv a = X
break
Print ("Inverse of b =", inv. b)

```

```

output:-
Enter prime modules P: 7
Enter First Integer a5
Enter" second. Integer, b:3
results:
Addition =: 1a
Subtraction= 2
multiplication 1
Inverse of a = 3
Inverse of b= 5

```

02 write a puthoni program to generate and print the-addition and multiplicaltable modulo aSmall prime
example: p=5

```

P= int(input("Enterca Smau prime : "))
Print (" " en'd = " ")
Print (f" In addition table mod fet:")
for b in range (D) :
Print (f" {b:20" end =""
for b in range (p) :0
Print (f" {[a b) % p:2}" end= "")
Jae Print Oiproorg Print(fiIn multiplication table mad {p1:") Print
("*" end. = "")n
for b in range (P):
Print Print (f {b:2} end= "")
print()
for a in range (P):
Print (f" a:2}", end= "")
for b in range (P):
Print (f "{(a*b) % P:2} " end= "")
print()

```

3. Write a python program to test whers Zn forms a Freld by Checking if is prime and report the result.
n = int (input ("Enter :n : ''))

```
Prime = True
if n<=1:
prime = False
for i in range(2, n):
prime:False
break
print(f"z_{n} forms field (since{n} is prime)")
else:
print(f"z_{n} does not form fields(since{n} is prime ).")
```